IGNOU – MASTER OF COMPUTER APPLICATIONS

MCSL-223: Computer Networks and Data Mining Lab

Assignment Answers (Simple Writing Format)

Course Code: MCSL-223

Assignment: MCA_NEW/MCAOL(II)/L-223/Assign/2026-27

Student Name: _____

Enrollment Number: _____

Study Centre: _____

Note: These answers are written in a simple format for assignment preparation. The NS-3 programs and WEKA steps are templates. Run them in your own environment and replace illustrative observations with your actual output before submission.

PART I – COMPUTER NETWORKS

Q1. Four-node network using Point-to-Point links

Aim: To create four nodes, assign IP addresses, install the Internet stack, and send packets from Node 1 to Node 4.

Topology: Node 0 — Node 1 — Node 2 — Node 3. Each adjacent pair is connected by a point-to-point link.

Program (NS-3, C++):

```
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"
using namespace ns3;

int main() {
  NodeContainer nodes;
  nodes.Create(4);

  PointToPointHelper p2p;
  p2p.SetDeviceAttribute("DataRate", StringValue("5Mbps"));
  p2p.SetChannelAttribute("Delay", StringValue("2ms"));

  InternetStackHelper internet;
  internet.Install(nodes);

  NetDeviceContainer d01 = p2p.Install(nodes.Get(0), nodes.Get(1));
  NetDeviceContainer d12 = p2p.Install(nodes.Get(1), nodes.Get(2));
  NetDeviceContainer d23 = p2p.Install(nodes.Get(2), nodes.Get(3));

  Ipv4AddressHelper ip;
  ip.SetBase("10.1.1.0", "255.255.255.0");
  ip.Assign(d01);
  ip.SetBase("10.1.2.0", "255.255.255.0");
  ip.Assign(d12);
  ip.SetBase("10.1.3.0", "255.255.255.0");
  ip.Assign(d23);

  Ipv4GlobalRoutingHelper::PopulateRoutingTables();

  UdpEchoServerHelper server(9);
  ApplicationContainer s = server.Install(nodes.Get(3));
  s.Start(Seconds(1.0)); s.Stop(Seconds(10.0));

  UdpEchoClientHelper client(Ipv4Address("10.1.3.2"), 9);
  client.SetAttribute("MaxPackets", UintegerValue(5));
  client.SetAttribute("Interval", TimeValue(Seconds(1.0)));
  client.SetAttribute("PacketSize", UintegerValue(512));
  ApplicationContainer c = client.Install(nodes.Get(0));
  c.Start(Seconds(2.0)); c.Stop(Seconds(9.0));

  Simulator::Run();
  Simulator::Destroy();
  return 0;
}
```

Procedure: Save the program as *q1.cc* in the NS-3 scratch folder and run it using the method supported by your NS-3 version. For example, newer versions commonly support *./ns3 run scratch/q1.cc*. The destination address 10.1.3.2 is assigned to Node 3 on the last link.

Expected observation: The UDP echo client sends packets to Node 3. The server replies, and the simulation output should show packet transmission and reception. If no reply appears, check IP addresses, routing, and application start times.

Result: A four-node point-to-point network was configured. Successful communication is verified when the echo replies are received.

Q2. Six-node network with at least three hops

Aim: To send packets across a six-node network and measure end-to-end delay and packet delivery.

Topology: Node 0 — Node 1 — Node 2 — Node 3 — Node 4 — Node 5. The route has five links (hops), which is more than the required three.

Program: Use the Q1 program as the base and change the node count to six. Install point-to-point devices on links (0,1), (1,2), (2,3), (3,4), and (4,5). Assign a different /24 subnet to each link, for example 10.1.1.0/24 through 10.1.5.0/24. Install the UDP echo server on Node 5 and the client on Node 0. Set the client destination to Node 5's address on the final link, usually 10.1.5.2, and populate routing tables.

Measurement: Enable FlowMonitor to obtain transmitted packets, received packets, lost packets, and delay statistics. Add these headers and code before Simulator::Run():

```
#include "ns3/flow-monitor-module.h"

// Before Simulator::Run():
FlowMonitorHelper flowHelper;
Ptr<FlowMonitor> monitor = flowHelper.InstallAll();

// After Simulator::Run(), before Simulator::Destroy():
monitor->CheckForLostPackets();
monitor->SerializeToXmlFile("q2-flow.xml", true, true);
```

Open *q2-flow.xml* or use the FlowMonitor statistics in the program. Average end-to-end delay is the total delay of received packets divided by the number of received packets. Packet delivery ratio is:

Packet delivery ratio (%) = (Packets received / Packets sent) × 100

Observation: Record the actual values reported by your simulation. For example, if 5 packets are sent and 5 are received, delivery ratio = (5/5) × 100 = 100%. This is an example calculation, not a claimed simulation result.

Result: A six-node route with at least three hops was designed. FlowMonitor can be used to measure delay and packet delivery.

Q3. Two paths between source and destination

Aim: To compare two paths with different link delays or bandwidths.

Topology: Path A: Node 0 — Node 1 — Node 3. Path B: Node 0 — Node 2 — Node 3. Use point-to-point links for all four connections.

Configuration: Give Path A a higher total delay (for example, 10 ms per link) and Path B a lower total delay (for example, 2 ms per link). Assign a separate subnet to each link, install the Internet stack, and call *Ipv4GlobalRoutingHelper::PopulateRoutingTables()*. Send UDP traffic from Node 0 to Node 3 and enable FlowMonitor.

Important note: Ordinary static/global routing does not always choose a path based on link delay or bandwidth. It may select a route based on routing metrics and topology. Therefore, do not assume that the lowest-delay path is automatically selected. To demonstrate path selection clearly, set route metrics explicitly or configure routes manually, then compare the measurements.

Experiment:

• Run 1: Configure Path A with a total delay of 20 ms and Path B with a total delay of 4 ms. Record the selected route and measured delay.

• Run 2: Change one link delay or data rate, then run the simulation again.

• Compare route selection, average delay, throughput, and packet loss in a small table.

Run	Changed setting	Route observed	Average delay	Packet loss
1	Initial values	Fill from output	Fill from output	Fill from output
2	Changed one link	Fill from output	Fill from output	Fill from output

Result: Two alternative paths were created. Changing link properties affects network performance; the selected route and measured values should be reported from the actual simulation.

Q4. Broadcast in a seven-node network and spanning tree

Aim: To observe broadcast propagation and compare it with a logical spanning tree.

Example topology: Connect nodes as follows: (0,1), (0,2), (1,3), (1,4), (2,4), (2,5), and (4,6). Node 0 is the source. This topology contains a cycle through nodes 0, 1, 4, and 2.

Method: Create seven nodes and install point-to-point links for the listed pairs. A normal UDP packet sent to one destination is unicast, not a network-wide broadcast. For a true broadcast experiment, implement a simple application that forwards a received broadcast once to each neighbour except the incoming interface, and use a packet ID or duplicate cache to avoid forwarding the same packet repeatedly. Alternatively, build a logical spanning tree and send one copy along each tree edge.

Spanning-tree example: Use edges (0,1), (0,2), (1,3), (1,4), (2,5), and (4,6). This connects all seven nodes using six edges and contains no cycle.

Comparison: A spanning tree connecting 7 nodes has exactly 7 – 1 = 6 edges. Thus, a single broadcast copy sent along each tree edge requires 6 transmissions. Flooding over all 7 physical links can require more transmissions, and careless forwarding may create duplicate packets or loops. The exact count depends on the forwarding algorithm and whether each

link direction is counted separately.

Result: A seven-node topology and its logical spanning tree were defined. A spanning tree reduces redundant forwarding by avoiding cycles while still reaching every node.

PART II – DATA MINING LAB

Q1(a). K-Means clustering on Customer Purchase dataset

Aim: To group customers with similar purchasing behaviour using K-Means in WEKA.

Sample CSV structure:

```
CustomerID,Age,Gender,AnnualIncome,SpendingScore,MembershipType,NumPurchases
1,22,F,25000,80,Basic,12
2,45,M,70000,35,Gold,5
3,31,F,48000,65,Silver,9
4,52,M,90000,25,Gold,4
5,26,M,30000,75,Basic,11
6,39,F,60000,50,Silver,7
```

The rows above are a small example dataset. For a meaningful clustering result, create a larger CSV file with more customers. CustomerID is an identifier and should generally not be used as a clustering feature. Gender and MembershipType are categorical, so encode them appropriately or exclude them in a simple numerical demonstration.

Steps in WEKA:

• Prepare the customer data in CSV format and open it in WEKA Explorer using Preprocess → Open file.

• Check missing values and correct attribute types. Remove CustomerID from the clustering features.

• Save as ARFF using Save, or use WEKA's CSV loader and save the loaded data as .arff.

• Go to Cluster → Choose → SimpleKMeans. Set numClusters to 3 as an initial choice and select the attributes to use.

• Click Start. Inspect the cluster assignments, centroids, and number of customers in each cluster.

Analysis: The clusters may represent high-spending customers, moderate-spending customers, and low-spending customers. This is only a possible interpretation; use the actual centroids and cluster membership to name the groups. Scale numeric features such as income and spending score if their ranges differ greatly.

Suitability of K: K = 3 is a reasonable starting point because it gives three easy-to-interpret customer groups. Compare K = 2, 3, and 4 and check whether clusters are meaningful and reasonably separated. Do not claim K = 3 is best unless the data supports it.

Result: K-Means can group customers by income, spending score, age, and purchase count. The best number of clusters depends on the dataset and the usefulness of the resulting groups.

Q1(b). K-Means clustering on Student Performance dataset

Aim: To group students according to attendance, assessment scores, assignment scores, and study hours.

Sample CSV structure:

```
StudentID,Attendance,InternalMarks,AssignmentScore,StudyHours,FinalGrade
101,92,82,90,12,A
102,65,55,60,5,C
103,85,75,80,10,B
104,58,48,52,4,D
105,95,88,94,15,A
106,76,68,72,8,B
```

Use a larger dataset for analysis. StudentID should be excluded from clustering because it is only an identifier. FinalGrade may be excluded if the goal is to discover groups from study behaviour and assessment measures, rather than using the final outcome as an input.

Steps: Open the CSV in WEKA Explorer; check and clean the data; save it as ARFF; choose Cluster → SimpleKMeans; set numClusters = 3; choose attendance, internal marks, assignment score, and study hours; click Start.

Analysis: The clusters may show students with consistently high performance, medium performance, and students needing support. Confirm these interpretations by inspecting the cluster centroids. Standardize numeric attributes when needed.

Suitability: Start with K = 3 for a simple high/medium/low grouping, then compare K = 2 and K = 4. Consider whether the clusters are easy to explain and useful for identifying support needs. The actual best K depends on the data.

Result: K-Means can reveal groups of students with similar learning and assessment patterns. The cluster labels are interpreted after examining the output; K-Means does not automatically know labels such as “good” or “weak” students.

Q2(a). Apriori on Retail Store Transactions

Aim: To find products that are frequently purchased together using the Apriori algorithm.

Sample transaction data:

```
TransactionID,CustomerID,Items,Category,Quantity,Day,PaymentMode
1,201,"Bread;Milk",Grocery,2,Monday,UPI
2,202,"Bread;Butter",Grocery,2,Tuesday,Cash
3,203,"Bread;Milk;Butter",Grocery,3,Wednesday,Card
4,204,"Milk;Butter",Grocery,2,Friday,UPI
5,205,"Bread;Milk",Grocery,2,Saturday,Cash
6,206,"Bread;Milk;Butter",Grocery,3,Sunday,Card
```

Apriori requires transaction data represented as items per transaction. Convert the data to a transaction format in which each product is a binary attribute (present/absent), or use a WEKA-compatible market-basket ARFF file. TransactionID and CustomerID are identifiers and should not be treated as purchased items.

Steps in WEKA:

• Prepare the transaction data and load it in WEKA Explorer.

• Represent each possible product as a nominal/binary attribute, with each row representing one transaction.

• Open Associate and choose Apriori.

• Set a minimum support value, for example 0.30, and minimum confidence, for example 0.70, where supported by the WEKA version.

• Click Start and record the frequent itemsets and association rules shown.

Example interpretation: A rule such as {Bread} ⇒ {Milk} means that transactions containing bread are associated with milk purchases. Confidence is the proportion of bread-containing transactions that also contain milk. Support is the proportion of all transactions containing both products.

For the six example transactions, Bread occurs in 5 of 6 transactions, Milk in 5 of 6, and Bread and Milk together in 4 of 6. Therefore, support for {Bread, Milk} = 4/6 = 66.7%. Confidence for Bread ⇒ Milk = 4/5 = 80%. These values are calculated from the illustrative data above; WEKA output may differ if you use another dataset.

Result: Apriori identifies frequent item combinations and rules that can help with product placement, promotions, and understanding shopping patterns.

Q2(b). Apriori on a market-basket dataset with different settings

Aim: To compare association rules generated with different minimum support and confidence values.

Procedure:

• Open a market-basket dataset in WEKA Explorer. Use a dataset with one row per transaction and item-presence attributes.

• Select Associate → Apriori.

• Run 1: Use minimum support = 0.30 and minimum confidence = 0.70. Record the rules.

• Run 2: Lower minimum support to 0.20 while keeping confidence at 0.70. Record the rules.

• Run 3: Keep support at 0.20 and raise confidence to 0.80. Record the rules.

• Compare the number of rules, support, confidence, and whether the rules are useful.

Setting	Expected general effect
Higher minimum support	Fewer rules; only more common item combinations remain.
Lower minimum support	More rules may appear, including less common combinations.
Higher minimum confidence	Fewer but stronger conditional rules may remain.
Lower minimum confidence	More rules may appear, but some are less reliable.

Interpretation: Support shows how frequently an item combination occurs in all transactions. Confidence shows how often the right-hand item occurs when the left-hand item occurs. A useful rule should have sufficient support and confidence and make practical sense. High confidence alone does not prove that one product causes another to be purchased.

Result: Apriori was applied conceptually to a market-basket dataset. Varying support and confidence changes the number and strength of generated rules. Add the exact rules and values from your own WEKA runs.

Conclusion

In Part I, point-to-point networks, multi-hop communication, alternative paths, and broadcast using a spanning tree were studied using NS-3 concepts. In Part II, K-Means was used to group customer and student records, while Apriori was used to discover frequent itemsets and association rules. Actual program output and WEKA screenshots or result tables should be added after running the experiments.

References

• IGNOU, MCSL-223 Assignment, July 2026–January 2027.

• NS-3 network simulator documentation and examples.

• WEKA Explorer documentation for SimpleKMeans and Apriori.